P2. Assignment 2.3 File protection with Digital Envelope -GradeScope-

Objective

In this week's lab, you will design and implement a dual cryptographic scheme to encrypt and decrypt files using a combination of symmetric cryptography, asymmetric cryptography and hashing algorithms. Your scheme must provide confidentiality, integrity, and authenticity ensuring that an attacker with access to the encrypted file is unable to read its contents and unable to modify the contents without being detected. The receiver must be able to verify that the sender encrypted the file.

Background

You will be using the OpenSSL toolkit for this lab. OpenSSL is a general purpose cryptographic toolkit that is widely used in both non-commercial and commercial applications. Read more about OpenSSL here.Links to an external site. Also read the man page of OpenSSL by typing man openssl in a terminal.

Here is a helpful video for bash scripting Link to videoLinks to an external site..

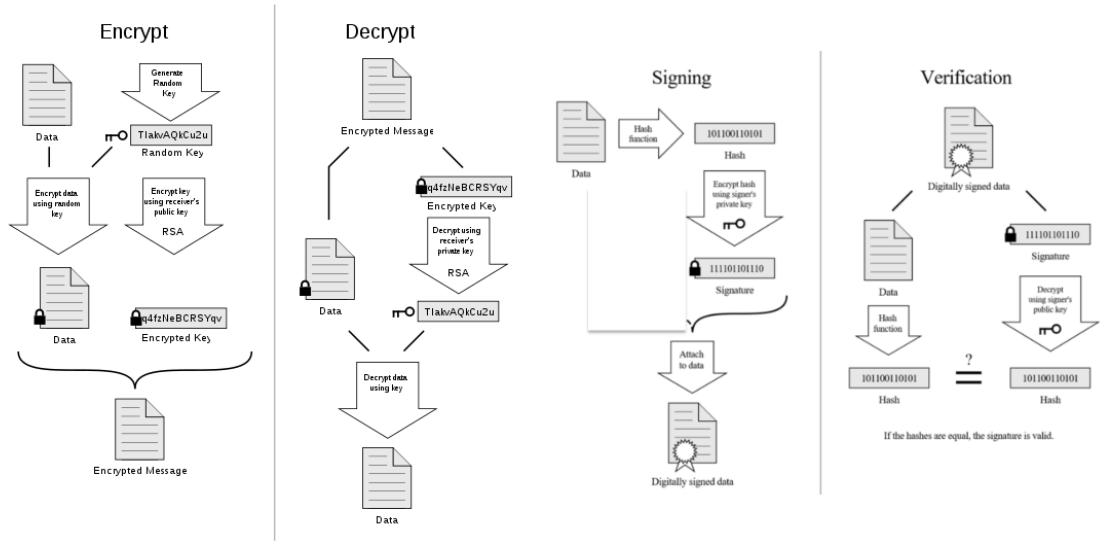
Setup

You should have OpenSSL already available in your VM, if you are using a different VM that the one we provided for the class, you may need to install it:

sudo apt-get install openssl

Instructions

- Design an efficient cryptographic scheme that takes advantage of the benefits of both, symmetric and asymmetric encryption. An implementation of this scheme must enable:
  1) (userid.sh) a sender to encrypt a file to be sent over email or by other means. On the other side, the receiver must be able to decrypt the file.
  2) (userid.sig.sh) a sender to sign a file to be sent over email or by other means. On the receiver side, the receiver must be able to verify the signature



- Write a bash script that implements this cryptographic scheme. You can generate these RSA key-pairs yourself by following the instructions in the previous assignment. Your encrypting script must output two files: one with the encrypted file (with extension .enc) and other the envelope with encrypted key (with extension .key). Remember that you need to generate a random session key every time the script is executed, so it needs to be different every time a file is encrypted.
- Do not hard code the name of script's argument files (neither filename nor extensions): input files or output files. These file names should be accepted as parameters when executing the script and will be different over the grading process.
- This lab is designed to be implemented in BASH using the OPENSSL version 3.0 or newer. You also need to follow the following MANDATORY implementation details:
-
    - Generate 2048 bit RSA key pair.
    - Symmetric Encryption using **aes-192-ctr, 1 million iter**ations as key derivation method, do not use salt (**-nosalt**).
    - Random session key generated inside your script using **openssl rand** and storing output into a file. Once the encryption is finished, make sure that you remove the random session key file and any other temporary files you used.

- Use the **pkeyutl** mode and **–enc**rypt option to create the envelope by encrypting the session key with the public key of the receiver.

-
  - All filenames and extensions cannot be hardcoded in the scripts, arguments could include fullpaths of the files (e.g.: /home/instructor/lab/decryptedfile.txt)
-
  - You need to imply the proper options and parameters to be used in your scripts.

1) userid.sh (encrypt and decrypt)

-
  - For encrypting a file:

./userid.sh -e <receiver_public_key_file> <plaintext_file> ***<encrypted_file>*** ***<encrypted_key_file>***

-
  - For decrypting the file:

./userid.sh -d <receiver_private_key_file> <encrypted_file> <encrypted_key_file> ***<decrypted_file>***

*Note: Script's **parameters in green** are output file names, and they are provided as argument to the script to determine **where script's output will be stored**.*

2) userid.sign.sh (signature and verify)

- Sign

./userid.sign.sh -s <private_key> <file_to_sign> <signature_output>

- Verify

./userid.sign.sh -v <public_key> <file_to_verify> <signature_file>

  - *Script's **parameters in green** are output file names, and they are provided as argument to the script to determine **where script's output will be stored**.*

- Use -sha256 -sign for signing and verifying
- Exit code 0 on valid signature, non-zero if verification fails.
- No hard-coded filenames.

Lab Submission

For this lab, each student must upload their script and test-files into Gradescope. Test files will be use to execute your script with the exact files you have you to test it. After, we will test your script with other files, which would have any name, so you cannot hardcode any name or extension of the arguments to your script.

1. The key pair(s) that you used to test your script. Name them ***<userid>.public.key.pub*** and **<userid>.private.key.priv** (being your username the one into your NEU email **userid**@northeastern.edu).
2. The ***<userid>.txt*** file, that you used to test your script.
3. Your scripts, named ***<userid>.sh*** and ***<userid>.sign.sh***.

This is an example of what Edmund Saw (saw.edm@northeastern.edu) submission looks like:

– saw.edm.sh
– saw.edm.sign.sh
– saw.edm.txt
– saw.edm.public.key.pub
– saw.edm.private.key.priv

Grading

Gradescope only checks certain aspects of your submission, hence this is just a provisional grade. Your script will be evaluated manually as well, and the grade may reflect the changes accordingly.

- 20 points - Encrypt/Decrypt with proper cryptographic scheme implementation in bash.
- 20 points - Sign/Verify with proper cryptographic scheme implementation in bash.
- 30 points - Scripts works with the provided student test files (<userid>.public.key.pub, <userid>.private.key.priv, <userid>.txt)
- 30 points - Scripts works with other plaintext, public and private key files.